Math 3709 Optimisation CW

Student ID:10724733

April 23, 2024

For an undirected graph G = (V, E), where V and E denote the sets of vertices and edges of the graph respectively, a vertex cover of G is a set of vertices that is a subset of V and includes at least one endpoint of every edge of G. The vertex cover problem is an NP-complete optimisation problem that aims to find a subset of V that is a vertex cover of the graph G, where the number of vertices in the solution is equal to a target number of vertices

History

The vertex cover problem is a variation of the minimum vertex cover problem, which aims to find the minimum number of vertices required to be a vertex cover, which is NP-hard. The origin of the vertex cover problem can be traced back to the beginning of graph theory in the 18th century. However, with the formation of the computational complexity theory, the vertex cover problem became a more applicable optimisation problem. It was studied in detail by Richard M Karp in 1972 and was listed among the 21 NP-complete problems in his paper "Reducibility among combinatorial problems" [2]. In this paper Karp demonstrated that the Vertex cover problem is a reduction of the Clique problem and thus the Boolean satisfiability problem (SAT), and hence is NP-complete [3].

Setup

Constructing the problem in python, a graph G can be interpreted using an adjacency matrix A to represent the vertices and edges of G as entries of the matrix. This can then be used to produce a visualisation of G.

To evaluate the problem, a trial graph G is needed from which to calculate the minimum subset V. Graphs of n vertices can be generated by first creating an n x n zero matrix O, and then randomly assigning some entries of the matrix to 1, while maintaining the diagonal elements as 0. Hence generating the symmetric adjacency matrix A for the graph G. This matrix is then used to construct the graph using the networkx library.

A genetic algorithm (GA) can be performed to solve the problem on the generated graph G. To test the effectiveness of the algorithm, the population size can initially be kept constant while the number of vertices of the generated graphs G can be varied. This will provide a baseline from which comparisons can be made with further modifications to the algorithm. Increasing the number of vertices should increase the difficulty of the problem, and hence provide a better understanding of the strength of the GA. To further test the algorithm, the hyperparameters (i.e. the crossover/mutation rates as well as the population size of the GA) can be modified to provide further insight into the effectiveness of the algorithm.

The fitness variable of any given solution in the solution space is given by the difference between the number of vertices in the individual and the target number of vertices k. This difference is not calculated as an absolute difference. This way the solutions can approach the target k (hence being a negative integer difference) and then surpass it (becoming a positive integer difference to the target k). This will be seen on the convergence plots for various instances. To ensure the solution space

maintains valid vertex covers the fitness of an individual with an invalid solution is interpreted as $+\text{inf}$, this way the algorithm should reject individuals with these fitness values. The target value used throughout the report will be $k = n^{0.7}$, where n is the number of vertices. This value was chosen to maintain the likelihood of possible solutions (i.e. avoid unsolvable instances) while still providing enough level of difficulty to test the algorithm. The reason for this target to be chosen is discussed later in the report.

Lastly, the mutation function of the algorithm simply replaces a randomly chosen individual in the solution space with a random arrangement of vertices from the original set of vertices V. This ensures the population is updated with new variations to prevent the solution space becoming stuck in a local minima.

Single Instance

Initial testing of the GA was conducted with unvaried values for the mutation and crossover rates. Population size was set to 200 and the algorithm was run for 500 iterations. An initial test of the algorithm can be performed on a single instance generated graph of 40 vertices with a target solution length of 30 vertices. Graphical illustration of the instance can be seen in fig. 1.

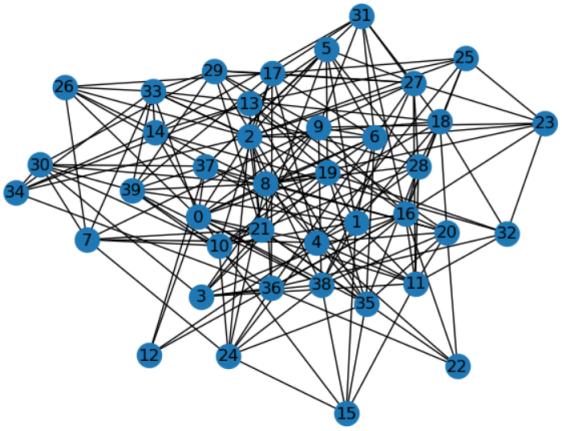


Figure 1: Graph G of 40 vertices

The algorithm solves this problem efficiently. The minimum possible vertex cover found was a solution space of 8 vertices as seen in fig. 2. Where the solution vertex set is highlighted in red. The efficacy of the algorithm can be seen in the convergence plot in fig. 3.

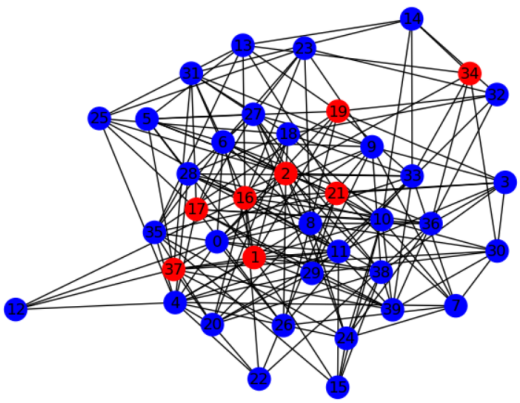


Figure 2: Solution Graph of G

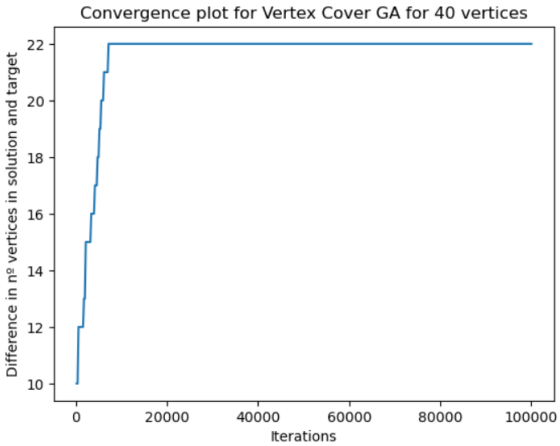


Figure 3: Convergence graph of fig. 2

The algorithm very quickly found a solution equal to the target number k and surpasses it. The GA rapidly betters upon this solution as seen by the large step increases at < 20000 evaluations before plateauing and the best fitness obtained (22). The algorithm finds no better solutions, which is likely due to it reaching the best possible solution.

Multiple Instances

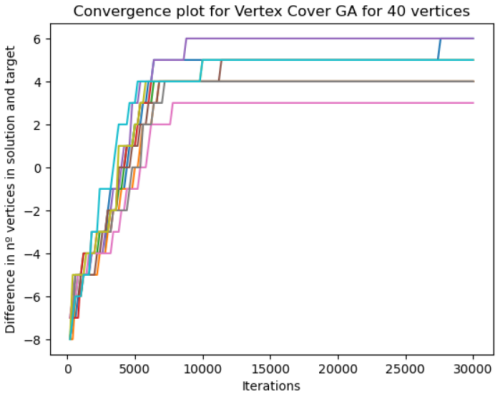


Figure 4: Converge graph for 10 instances

By using the same method as in the single instance case but with the number of iterations reduced to 150, the number of instances can be increased for the same number of vertices to observe the frequency of convergence of the algorithm. The number of iterations was reduced considering the findings for the single instance case. fig. 4 illustrates the convergence graph for 10 separate instances of the algorithm attempting to solve the vertex cover problem. The problem variables were set to 40 vertices, with a target of $k = 13$. In all cases the algorithm converged, the weakest of which is seen by the pink line, which found a solution of $k - 3$ vertices (10 vertices). The strongest convergence was $k - 6$ (7 vertices).

It should be noted that for some generated graphs, a solution space may not exist according to the desired target, and hence the algorithm won't discover any solutions. This effect is more notable for very small number of vertices ($n < 3$), as the chance of generating a disconnected graph is highly likely and hence finding no solutions at a target of $k = n^{0.7}$. There is also inconsistency with the interpretation of results from the convergence graphs. Due to all test graphs being randomly generated, the level of difficulty in finding solutions will vary between instances. To combat this the number of instances for each problem should be maintained at a sufficient level as to remove any ambiguity about the convergence of the GA.

Convergence

To analyse the frequency of convergence of the algorithm we vary the target k from 2 to 20 in increments of 2 and record the frequency of reaching a vertex cover within the target amount over 3 instances per vertex quantity, hence running the algorithm for a total of 30 instances. The vertex count was set to 20 as the size of the graph is nearly negligible when only considering if the target is met. The GA was set to perform 150 generations/iterations on each instance. fig. 5 illustrates the results of this test. Out of 3 instances per graph size, the y axis represents how many failed to converge. The x-axis represents the target in the instance. It is clear that the algorithm converges with certainty for $k > 5$. The vertical red line at $k = 8$ is equivalent to the target values used throughout the report of $k = n^{0.7}$.

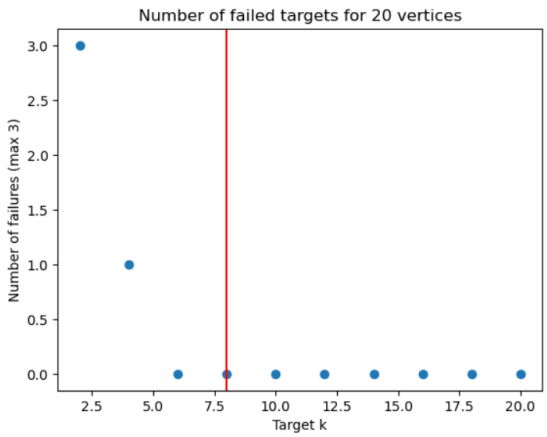


Figure 5: Fail count for varying graph size

Fig.5. validates the use of this value as a suitable choice.

As mentioned before, due to the unpredictability of the randomness in the test graphs generated, it is impossible to be certain if all the instances are unsolvable. In the later section of the report we consider graphs with known solutions to remove the uncertainty. Across the convergence plots in Figures 3 and 4 the convergence occurs rapidly, indicating that the increasing the number of iterations needed to find an optimum solution should have no effect under its current parameters. For larger graph sizes however, more iterations may be required to reach a solution. As a result, modifying hyperparameters could improve the effectiveness of the algorithm.

Hyperparameter Modification

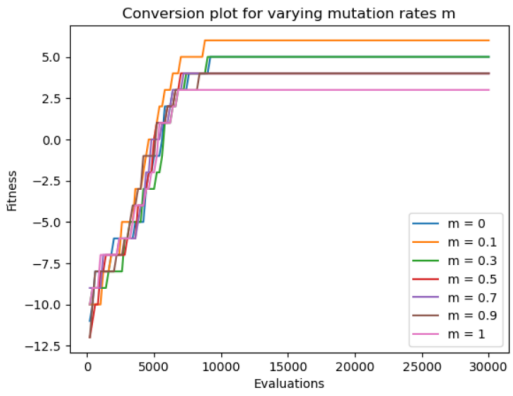


Figure 6: Convergence of varying mutation

To test the effects of varying the hyperparameters of the algorithm, each modification will be applied to the same graph of 50 nodes. Across all tests, the target value is $k = n^{0.7}$, as before. This should allow for targets to be met in most cases by avoiding unsolvable targets. The first modification is applied to the mutation rate hyperparameter. The entire range of mutation probability m was tested in increments (0%, 10%, 30%, ..., 90%, 100%). fig. 6 represents the results of this test.

From the graph it is clear that the effects of changing the mutation rate are minimal. All mutation rates converged in the same manner with the same efficiency, however the algorithm found the best solution when the mutation rate was set to 10%. The fitness of the results seems to deteriorate as the mutation rate increases, suggesting that the mutation rate should be kept to a minimum, but not at 0.

The crossover hyperparameter c can also be modified to analyse its impact on the effectiveness of the GA. The rates were varied in the same range as in the mutation investigation except for the removal of the 0% case. The results as seen in fig. 7 indicate that convergence improved for more extreme crossover rates. It seems that lower crossover rates converge slower, although the difference is insignificant.

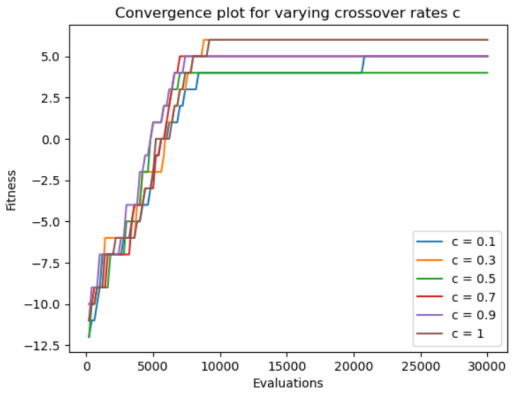


Figure 7: Convergence of varying crossover

A final hyperparameter modification can be performed on the population size p of the GA. The same problem conditions were given as in the previous two cases. The population sizes considered are as indicated in the legend of fig. 8. There is a clear correlation between the population size as the fitness obtained by the GA. This is to be expected as the GA's mutation is solely based on random generation, hence the more mutations occurring the higher the likelihood of generating a more

optimum solution.

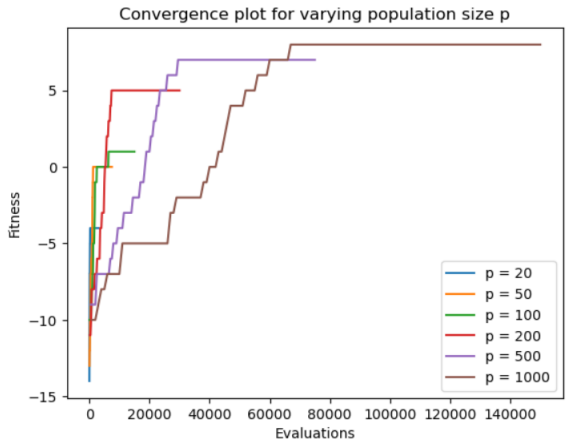


Figure 8: Convergence of varying population size

The results of the hyperparameter modification tests suggest that the population size should be maximised, the mutation rate should be kept to approximately 0.1, and the crossover probability should be increased but limited as a high crossover rate could lead to omitted solutions.

Algorithm Limit Investigation

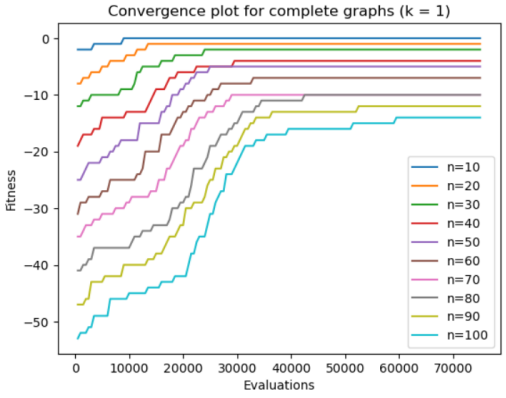


Figure 9: Convergence of complete graphs

So far, the testing conducted has been on randomly generated graphs with a randomly assigned number of edges (50% chance for any two vertices to be connected). To better understand the convergence of the GA the algorithm was tested on graphs with known solutions. The simplest example of graphs with known vertex cover solutions are complete graphs (all vertices connected).

The GA was modified according to the conclusions of the hyperparameter analysis for 150 iterations (population size = 500, mutation rate = 0.1, crossover rate = 0.8). The target value in all cases was set to 1. The results of the analysis highlight the pattern of convergence for the algorithm. If the vertex count is low enough ($n < 20$), the GA rapidly finds the minimum solution. For vertex counts $n > 20$, the GA cannot find the minimum solution, with the difference between the minimum solution and the best solution found increasing linearly as the vertex count increases. This conclusion is further supported by fig. 10. As the graphs generated are complete, they have a known solution size of 1.

Comparing this with fig. 10 there is a clear linear relationship between the size of the graph and the minimum solution size found.

Although the target in these cases is of the minimum possible size (k = 1), this will not be the case for most other graphs, and as shown by the convergence test in fig. 5 as long as the target is in the range $k \geq n^{0.7}$ then the GA will find a viable solution. The results of fig. 10 suggest that the solution will not be the minimal solution.

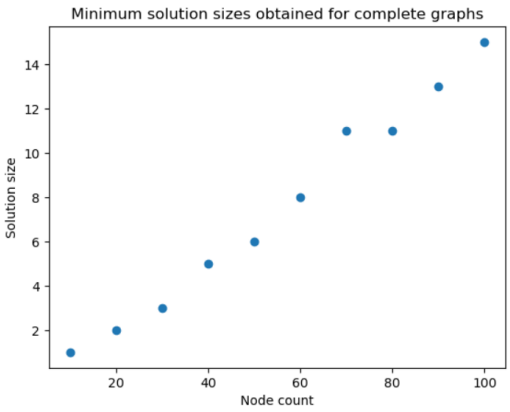


Figure 10: Solution size obtained

Time Analysis

A final analysis can be performed on the time taken to perform the GA given the vertex count for a constant number of iterations. A higher mutation count/ lower crossover rate is shown to increase the time taken to run a GA; hence they will be set to the default values to avoid bias.

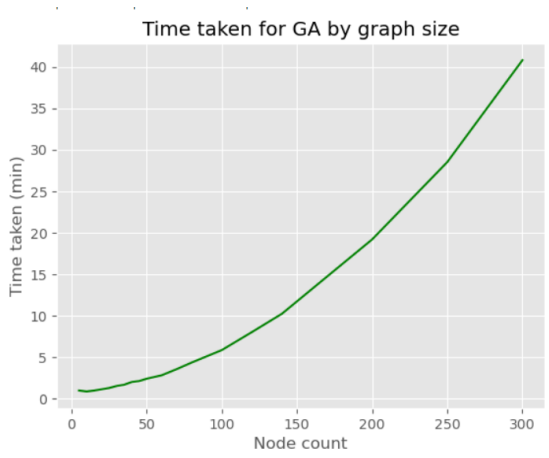


Figure 11: Time taken for varying size

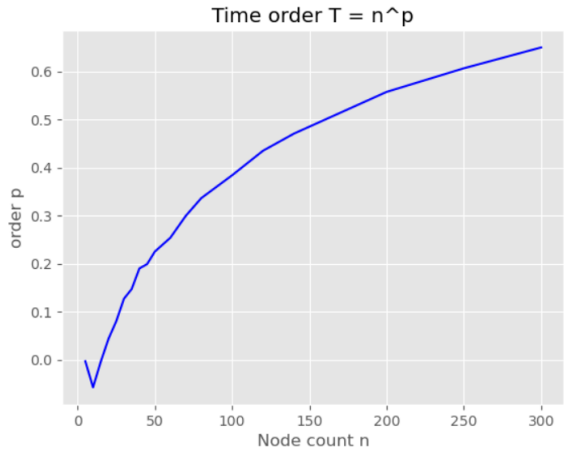


Figure 12: Order p for varying size

The GA was run for 300 iterations per node count to allow for a significant time comparison between instances. fig. 11 presents the results of the analysis. The time taken to solve 300 iterations of a given instance increases seemingly exponentially as the node count increases. The graph clearly indicates the NP-complete property of the vertex cover problem; the problem cannot be solved in polynomial time. By assuming the time t can be expressed as a polynomial in the node count n; $t = n^p$. Using the values shown by fig. 11 we can solve $p = log_n(t)$, and thus the order can be plotted against the node count, as seen by fig. 12. There is no consistent order against the node count, hence confirming the non-polynomial time property of the vertex cover problem.

Discussion

The vertex cover problem is important in mathematics and graph theory as it is not only crucial in many real-world applications (e.g. resource allocation), but its solution could also reveal the solutions to further optimisation problems connected to it, as well as providing a better overall understanding of computational complexity theory. The genetic algorithm used to solve the vertex cover problem here is very basic, and far from the most powerful algorithm such as those modernly used, e.g. the approximation used by Harris, D.G. and Narayanaswamy, N.S. (2024), where solutions were calculated to $O(1.25284^n)$ [1]. Furthermore, meta-heuristic approaches have also found success but still to a non-polynomial order in time [4] The analyses performed could be improved by using only graphs with known solutions, however, the results of these tests would likely produce the same conclusions as have been derived, but with greater certainty.

The tests performed throughout illustrate the capabilities of the basic GA used. For a graph of n vertices, a solution can be found as long as the chosen target k satisfies $k \geq n^{0.7}$. Without satisfying this uncertainty the viability of the GA finding a solution is solely based on the complexity of the graph generated. Likewise, if the graph generated has a high complexity (large n) then the chance of the algorithm finding the minimum possible solution becomes less linearly less likely.

In conclusion, the vertex cover problem is a NP-complete problem that is useful to investigate as its results provide a deeper insight into the intricacies of computational complexity theory and can help in the practical development of further computational techniques.

References

[1] Harris, D.G. and Narayanaswamy, N.S. (2024). A faster algorithm for Vertex Cover parameterized by solution size. [online] arXiv.org. https://doi.org/10.48550/arXiv.2205.08022

[2] Karp, R.M. (1972). Reducibility among Combinatorial Problems. Complexity of Computer Computations, pp.85–103.https://doi.org/10.1007/978-1-4684-2001-2_9

[3] Cook, S.A. (1971). The complexity of theorem-proving procedures. Proceedings of the third annual ACM symposium on Theory of computing - STOC '71.https://doi.org/10.1145/800157.805047

[4] Mehrabi, A.D., Mehrabi, S. and Mehrabi, A. (2009). A PRUNING BASED ANT COLONY ALGORITHM FOR MINIMUM VERTEX COVER PROBLEM. [online] www.scitepress.org.https://doi.org/10.5220/0002313802810286